

Now tracking all four Claude certifications. Architect (Foundations) stays free and fully open. The Professional, Associate, and Developer tracks are on their way — each has a new page covering what its exam tests. [Send feedback](#) on what to build first. Thanks, Walter

DISMISS

RESULTS

Exam Results

609 / 1000

NOT PASSED

36 of 60 correct (60%) · Pass mark: 720

Domain Breakdown

D1	Agentic Architecture & Orchestration	11/14 (79%) +212pts
D2	Tool Design & MCP Integration	5/11 (45%) +82pts
D3	Claude Code Configuration & Workflows	11/12 (92%) +183pts
D4	Prompt Engineering & Structured Output	3/12 (25%) +50pts
D5	Context Management & Reliability	6/11 (55%) +82pts

HOW SCORING WORKS

Each domain's percentage correct is multiplied by its weight and scaled to 1000 points. For example, scoring 80% in Agentic Architecture (27% weight) contributes $0.80 \times 0.27 \times 1000 = 216$ points. The pass mark is 720/1000 (72%). If a domain has no questions in the selected scenarios, its weight is redistributed proportionally among assessed domains.

The real exam reports a scaled score from 100 to 1,000 plus your percent-correct by domain, so treat the percentages here as a readiness gauge rather than a prediction of your scaled score.

HIDE DETAIL

TAKE ANOTHER EXAM

Question-by-Question Review

ALL 60

CORRECT 36

INCORRECT 24

SKIPPED 0

Q01 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ERROR-CONTEXT q-2-2-011 · Multi-Agent Research System

CORRECT

A research subagent's database query fails partway through an investigation. Which pieces of information belong in the structured error context it returns to the coordinator? (Select 3)

A No report at all: the subagent should retry silently until the query succeeds.

Silent retries hide failures from the coordinator, which owns error handling and needs observability into what went wrong.

B Partial results gathered before the failure.

Partial results preserve completed work so the coordinator does not re-run what already succeeded.

C The failure type, such as transient, validation, or permission.

Categorising the failure lets the coordinator make the right retry-or-reroute decision.

D A plain 'no results found' message in place of the error.

Reporting an access failure as an empty result conflates two different situations and hides the need for a retry decision.

E What was attempted, including the query that failed.

The attempted query gives the coordinator enough detail to retry intelligently or reformulate the request.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Four error categories)

Lesson 2.2: Structured Error Responses (Access failure vs empty result)

MCP: Tools Specification ↗

Q02 · D1 1.6 TASK-DECOMPOSITION / ADAPTIVE-DECOMPOSITION q-1-6-009 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

You are planning how Claude should tackle a large, unfamiliar refactoring effort. Which characteristics of the work indicate dynamic adaptive decomposition rather than a fixed sequential pipeline? (Select 2)

A Each step's output feeds the next in a stable, known order.

A stable, known step order is the defining property of a sequential pipeline, not of adaptive decomposition.

B The review covers the same predictable set of aspects on every run.

A predictable multi-aspect review is exactly where a fixed prompt-chaining pipeline fits best.

C The useful subtasks only become clear as intermediate findings come in.

Adaptive plans generate subtasks from what each step discovers, which a fixed pipeline cannot do.

D The task is open-ended, like adding comprehensive tests to a legacy codebase you have not mapped yet.

This is the guide's example of open-ended work: map the structure first, then build a prioritised plan that adapts as dependencies surface.

WHERE THIS COMES FROM

Lesson 1.6: Task Decomposition Strategies (Dynamic adaptive decomposition)

Lesson 1.6: Task Decomposition Strategies (Sequential pipelines)

Q03 · D3 3.4 PLAN-MODE-EXECUTION / PLAN-MODE q-3-4-006 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

A developer extracting the notification subsystem from a monolith faces 12 cross-module

dependencies, 3 messaging patterns, and several valid extraction strategies. They start in direct execution mode. After moving 8 files, the chosen approach breaks circular dependencies with the user-profile module. What should they have done differently?

- A** Used plan mode to map the 12 dependencies and evaluate extraction strategies before committing

A subsystem with 12 dependencies, multiple messaging patterns, and several valid strategies is a textbook plan mode scenario. Plan mode would have mapped the dependency graph and revealed the circular dependency with user-profile before any files were moved, avoiding costly rework.

- B** Used direct execution but with more detailed upfront instructions specifying how to handle each dependency

The developer did not know about the circular dependency with the user-profile module upfront. More detailed instructions cannot account for undiscovered dependencies. The codebase needed exploration first.

- C** Used direct execution but processed only 2 files at a time to catch problems earlier

Smaller batches in direct execution do not address the fundamental issue: the developer chose a strategy without understanding the full dependency landscape. The circular dependency would still be discovered mid-migration, just slightly sooner.

- D** Delegated the entire extraction to a subagent to isolate the risk

Delegating to a subagent does not change the outcome if the subagent also uses direct execution without exploring dependencies. The mode of execution (plan vs direct) matters more than who executes it.

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Plan mode for migrations)

Lesson 3.4: Plan Mode vs Direct Execution (Recognising complexity)

Q04 · D2 2.2 STRUCTURED-ERROR-RESPONSES / BUSINESS-ERROR q-2-2-002

CORRECT

A support agent tries to refund \$850, but policy caps automated refunds at \$500. The MCP tool returns 'Operation failed' as its error. The agent retries three times before escalating to a human. What change to the tool's error response would most effectively prevent this behaviour?

- A** Return a structured error with `errorCategory: 'business'`, `isRetryable: false`, and a customer-friendly description explaining the policy limit.

Business errors are never retryable — retrying a policy violation will always fail. Structured metadata with `errorCategory` and `isRetryable: false` tells the agent to stop retrying and take an alternative path, such as offering partial refund or escalating to a supervisor.

- B** Catch the error in the agent's system prompt with a standing instruction telling it never to retry a refund operation once that operation has failed.

Hard-coding retry rules per operation in the system prompt is brittle and does not scale. The proper fix is structured error metadata that the agent's recovery logic can interpret programmatically for any tool.

- C** Set the MCP `isError` flag and add a `retry-after` header so the agent waits before retrying.

A `retry-after` header implies the error is transient and will resolve with time. A business policy violation is never transient — no amount of waiting will change the refund limit. This would cause delayed retries that still fail.

- D** Include a longer error message explaining the policy: 'Operation failed because the refund amount of \$850 exceeds the \$500 automated refund limit'.

A better message helps human readers but does not give the agent structured metadata to determine that retrying is futile. The agent needs machine-readable fields, not just better prose.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Error categories)

Q05 · D3 3.2 SLASH-COMMANDS-SKILLS / ALLOWED-TOOLS-SCOPING q-3-2-012 ·

Code Generation with Claude Code

CORRECT

A junior developer is using Claude Code to refactor a payment processing module. The team lead wants to ensure Claude Code cannot accidentally delete production configuration files or modify the database migration directory during the refactoring session. What is the most appropriate approach?

- A Add a rule in CLAUDE.md stating 'Never modify files in the config/ or migrations/ directories'

CLAUDE.md instructions rely on the model's compliance and are not deterministic safeguards. For production-critical protection, tool-level permission restrictions provide reliable enforcement rather than natural language instructions.

- B Set the whole repository to read-only at the filesystem level before the refactoring session starts.

Making the entire repository read-only would prevent Claude Code from making any changes, including the legitimate refactoring of the payment module. The requirement is selective protection, not blanket read-only access.

- C Instruct the developer to use plan mode so Claude Code will ask for approval before each file change

Plan mode helps with evaluating strategies for complex tasks, but it does not provide file-level access control. Even in plan mode, the model can still access all tools and directories once execution begins.

- D Use allowedTools to limit the session to Read, Grep, Glob, and Write within the payment module

allowedTools provides tool-level permission control that can restrict which tools are available and their scope. By limiting write access to the payment module directory and keeping read access for context, the team lead ensures Claude Code cannot modify production config or migration files.

WHERE THIS COMES FROM

Claude Code: Settings ↗

Q06 · D5 5.6 INFORMATION-PROVENANCE / CONFLICT-HANDLING q-5-6-001 · Multi-Agent Research System

INCORRECT

A multi-agent research system produces a synthesis report on market trends. Two credible sources report different growth rates for the same sector: Source A reports 12% growth (2023 data) and Source B reports 8% growth (2024 data). The synthesis agent currently selects the more recent value. What is the correct approach?

- A Always use the most recent of the two sources, on the grounds that it reflects the latest data.

This silently discards valid historical context. The difference may reflect an actual trend, not a contradiction.

- B Average the two values and report 10% growth, with a note recording the variance between sources.

Averaging conflicting statistics is mathematically misleading and destroys the temporal context that explains the difference.

- C Flag the conflict and escalate to a human researcher for resolution before including in the report

This is unnecessary — the values are not contradictory, they reflect different time periods. Proper annotation resolves the ambiguity. (your answer)

- D Annotate both values with source and publication dates, letting the consumer interpret them.

This preserves provenance, temporal context, and both data points. The consumer can see that different dates explain different numbers. (correct answer)

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflict handling)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Temporal awareness)

Q07 · D2 2.1 TOOL-SCHEMA-DESIGN / DESCRIPTIONS q-2-1-011

INCORRECT

Your federated query platform exposes several similar search tools, and Claude keeps routing requests to the wrong one. Which changes improve tool selection reliability? (Select 3)

- A Trim every description to one short sentence so the model relies on tool names alone.

Minimal descriptions cause unreliable selection among similar tools; names alone cannot carry boundary information. (your answer)

B Rename tools whose names overlap so each name reflects a distinct function.

The guide's example renames `analyze_content` to `extract_web_results`, removing the functional overlap that caused misrouting.

C Merge the similar tools into one generic tool with a mode parameter.

The guide moves in the opposite direction: split generic tools into purpose-specific tools with defined contracts.

D Rewrite each description to state the tool's purpose, inputs and outputs, and when to prefer it.

Descriptions are the primary signal Claude uses to choose tools; differentiating them is the first fix for misrouting.

E Check the system prompt for keyword-sensitive instructions that create unintended associations with particular tools.

System prompt wording can override well-written descriptions, so it belongs in the same review. (correct answer)

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (Misrouting)

Lesson 2.1: Tool Interface Design (Dialect-specific descriptions)

Anthropic: Tool Use Documentation ↗

Q08 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ACCESS-VS-EMPTY q-2-2-008

CORRECT

The data platform's `fetch_api` tool calls a third-party pricing API. When the API key has expired, the tool returns `{ "data": [], "status": "ok" }` instead of signalling an authentication failure. The agent tells the user 'No pricing data is available for that product' and moves on. What is the root cause?

A The third-party API is poorly designed. The platform team should switch to a different pricing provider that returns proper HTTP error codes.

Whilst the upstream API behaviour is unhelpful, the platform team controls the MCP tool layer. The tool should detect the authentication failure (even from the upstream response) and translate it into a structured MCP error rather than passing through the misleading response.

- B** The agent's context window is too small to hold the full API response, so it receives a truncated version that appears empty.

Context window truncation would not produce a well-formed { "data": [], "status": "ok" } response. The problem is semantic — the tool is returning a misleading success status for an authentication failure.

- C** The agent should be instructed via the system prompt to treat empty arrays as errors and retry with exponential backoff.

Not all empty arrays are errors — a product with no pricing data is a valid empty result. The problem is that the tool masks an authentication failure as a successful empty response. System prompt instructions cannot reliably distinguish the two.

- D** The tool cannot distinguish an access failure (expired key) from a valid empty result, so the agent treats a permission error as no data.

The tool returns the same structure for two fundamentally different outcomes: 'I could not access the data' versus 'I accessed the data and found nothing.' The agent cannot distinguish these and accepts the empty result at face value. The tool must return a structured error with the MCP `isError` flag for authentication failures.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Access failure vs empty result)

Q09 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ERROR-CATEGORIES q-2-2-006 · Multi-Agent Research System

INCORRECT

A `search_papers` MCP tool has three failure patterns: (1) the upstream academic API returns HTTP 503 during peak hours, (2) users request papers from a restricted journal the system has no licence for, and (3) the agent submits a malformed DOI like 'doi-1234' that fails the input regex. The team wants structured error metadata so agents can handle each case differently. Which `errorCategory` and `isRetryable` combination is correct for all three?

- A All three should be errorCategory: 'transient', isRetryable: true, since they all prevent the tool from completing its task.

Lumping every failure into 'transient' is the canonical anti-pattern. The restricted journal case is a business error (licence policy); the agent must escalate, not retry. The malformed DOI is validation; the agent must repair the input format first.

- B HTTP 503: transient/retryable; restricted journal: transient/retryable; malformed DOI: transient/retryable.

The restricted journal error is not transient — the system lacks a licence, and no amount of retrying will grant access. A malformed DOI is validation, not transient; the agent must repair the input rather than blindly retry the same value.

- C HTTP 503: transient/retryable; restricted journal: business/not retryable; malformed DOI: validation/retryable.

HTTP 503 is a temporary upstream outage (transient, will likely resolve on retry). Restricted journal is a policy limitation (business, will never resolve on retry — escalate or offer an alternative source). Malformed DOI is validation; the agent must repair the input and then retry. (correct answer)

- D HTTP 503: validation/retryable; restricted journal: business/not retryable; malformed DOI: business/not retryable.

HTTP 503 is transient (a temporary upstream outage), not validation — nothing is wrong with the request input. A malformed DOI is validation (fix input and retry), not business — business errors represent policy or rule violations, not input format problems. (your answer)

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Four error categories)

Q10 · D1 1.5 AGENT-SDK-HOOKS / POSTTOOLUSE-NORMALISATION q-1-5-007 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

INCORRECT

During the monolith-to-microservices refactoring, each extracted service must follow a consistent Java package naming convention (com.company.service.<service-name>). The team notices that subagents sometimes use inconsistent package names like com.company.app.<service-name> or com.company.<service-name>. A team member proposes adding stronger instructions to the system prompt. What is the correct approach?

- A Create a separate validation subagent that reviews all written files for naming compliance after each refactoring batch

A validation subagent is still probabilistic — it may miss violations. PostToolUse hooks provide deterministic, inline enforcement that catches every violation at the point of file creation. (your answer)

- B Use tool_choice to restrict the agent to a custom write_java_file tool that enforces the naming convention in its implementation

Creating a custom tool wrapper adds unnecessary complexity. PostToolUse hooks on the existing file write tool achieve the same deterministic enforcement without modifying the tool interface.

- C Implement a PostToolUse hook that inspects written files and normalises any package declarations to the correct com.company.service.<service-name> format

A PostToolUse hook on file write operations can inspect the output and normalise package names deterministically. This provides a 100% enforcement guarantee regardless of what the model generates, which is the correct approach for structural consistency rules. (correct answer)

- D Add the naming convention to the system prompt with three concrete examples showing the correct pattern

Prompt instructions with examples improve consistency but still have a non-zero failure rate. Package naming is a deterministic rule that must be enforced 100% of the time to avoid broken imports across services.

WHERE THIS COMES FROM

Lesson 1.5: Agent SDK Hooks (PostToolUse normalisation)

Anthropic: Agent SDK Hooks ↗

Q11 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / PARALLEL-SPAWNING q-1-3-002 ·

Multi-Agent Research System

CORRECT

A multi-agent research system has a coordinator that spawns a web search subagent and a document analysis subagent sequentially across separate API turns. The web search completes in 8 seconds and the document analysis completes in 12 seconds, giving a total latency of 20 seconds. The two subagents are investigating independent topics and do not depend on each other's results. How should the architect reduce this latency?

A Merge the web search and document analysis into a single subagent to reduce coordination overhead

Merging agents violates the principle of specialised subagents with scoped tool access. It also does not reduce total processing time — the merged agent still needs to do both tasks, and loses the benefit of parallel execution.

B Use `fork_session` to split the coordinator into two parallel branches, one for each subagent

`fork_session` is for divergent exploration of the same problem from a shared baseline, not for parallel subagent invocation. Emitting multiple Task tool calls in a single coordinator response is the correct mechanism for spawning concurrent subagents.

C Allow the web search subagent to directly invoke the document analysis subagent after it finishes, removing the coordinator round-trip

Direct subagent communication violates the hub-and-spoke architecture where all communication flows through the coordinator. This also would not achieve parallel execution — it would still be sequential.

D Have the coordinator emit both Task tool calls in a single response to spawn both subagents in parallel, reducing total latency to roughly 12 seconds

When subagents investigate independent topics, the coordinator can emit multiple Task tool calls in a single response. Both subagents run concurrently, so total latency is determined by the slower one (12 seconds) rather than the sum (20 seconds).

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning)

Lesson 1.3: Subagent Invocation and Context Passing (Task tool)

Q12 · D2 2.1 TOOL-SCHEMA-DESIGN / BOUNDARY-DESCRIPTIONS q-2-1-006

INCORRECT

A platform has ``search_knowledge_base`` ('Searches help articles') and ``process_action`` ('Handles customer actions like refunds and plan changes'). The agent picks ``search_knowledge_base`` for 'cancel my subscription' requests. After adding 'subscription cancellations' to ``process_action``'s description, it now picks ``process_action`` for knowledge queries like 'how does cancellation work?'. What is the most effective solution?

- A Consolidate search_knowledge_base and process_action into a single tool that determines the correct action based on user intent.

Consolidation pushes the routing problem inside the tool implementation and loses the LLM's natural language understanding. Keeping tools separate with clear boundary descriptions is simpler and more maintainable.

- B Add boundary descriptions to both tools that separate executing a cancellation from learning how cancellation works.

Boundary descriptions resolve the ambiguity by intent. process_action should say to use it when the customer wants to execute a cancellation; search_knowledge_base should say to use it when the customer wants to learn about the process without taking action. Stating both boundaries prevents cross-routing in either direction. (correct answer)

- C Remove the word 'cancellation' from both tool descriptions to eliminate the keyword conflict entirely.

Removing relevant keywords makes both tools less discoverable for cancellation-related queries. The solution is to differentiate when each tool handles cancellation, not to hide the concept from both.

- D Add few-shot examples to the system prompt showing five cancellation scenarios with the correct tool for each.

Few-shot examples consume tokens and do not scale to the many variations of cancellation queries. Boundary descriptions in tool definitions are more efficient and generalise better. (your answer)

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (Boundary descriptions)

Lesson 2.1: Tool Interface Design (Misrouting)

Q13 · D1 1.7 SESSION-STATE-RESUMPTION / FORK-SESSION q-1-7-002 · Multi-Agent Research System

CORRECT

A research team is using Claude Code to analyse a large dataset. After completing an initial analysis, they want to explore two competing hypotheses: one using a statistical modelling approach and another using a machine learning approach. Both explorations should start from the same baseline analysis but proceed independently. Which session management strategy is

correct?

- A Resume the session twice with `--resume`, once for each hypothesis, running them one after the other

Resuming the same session twice would mean the second resume inherits context from the first hypothesis exploration, contaminating the independent baseline. Each hypothesis needs to diverge from the original analysis, not from each other.

- B Start two fresh sessions, each with an injected summary of the initial analysis, and explore one hypothesis in each

While this avoids context contamination, it loses the full richness of the original analysis context. Fresh start with summary injection is best when tool results are stale. Here, the baseline analysis is still valid and both branches should start from the complete shared baseline.

- C Use the initial session and explore both hypotheses sequentially, asking the agent to set aside the first approach before starting the second

Sequential exploration in the same session means the second hypothesis is influenced by the first — the agent carries context, conclusions, and biases from the first exploration. Independent exploration requires isolated branches.

- D Use `fork_session` to create two independent branches from the shared analysis baseline, exploring one hypothesis in each fork

`fork_session` creates independent branches from a shared baseline, which is exactly the use case here. Both forks start from the complete initial analysis but proceed independently. Neither exploration contaminates the other, enabling a clean comparison of the two approaches.

WHERE THIS COMES FROM

Lesson 1.7: Session State and Resumption (Session management options)

Lesson 1.3: Subagent Invocation and Context Passing (`fork_session`)

Anthropic: Claude Code Documentation ↗

Q14 · D4 4.3 STRUCTURED-OUTPUT / ENUM-FIELDS q-4-3-004 ·

Content Moderation and Classification System

INCORRECT

The moderation system's classification schema has a 'category' field defined as a free-text

string. Auditors find 47 different category values in production data, including 'hate speech', 'Hate Speech', 'hate-speech', 'hateful content', and 'hate_speech' — all intended to be the same category. This makes downstream analytics and routing unreliable. What is the best schema fix?

- A Add detailed instructions to the prompt listing the exact category names and their capitalisation so the model always emits the canonical string.

Prompt instructions are probabilistic. The model may still produce variations despite instructions. Schema-level enforcement via enums is deterministic and cannot be overridden.

- B Change 'category' from free-text to an enum with values like 'hate_speech', 'spam', and 'harassment', plus an 'other' option.

Enum fields constrain the model to predefined values, eliminating spelling and formatting variations. Including 'other' as an enum value handles edge cases without allowing free-text drift. With strict: true, the schema guarantees only valid enum values are returned. (correct answer)

- C Add a post-processing normalisation step that maps all variations to canonical category names

Post-processing normalisation adds complexity and requires constant maintenance as new variations appear. The schema should prevent the problem at the source rather than patching it downstream. (your answer)

- D Add few-shot examples showing the correct category formatting for each type

Few-shot examples improve consistency but cannot guarantee exact string matching. Schema enums are the correct mechanism for constraining categorical output to a fixed set of values.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (Enum schema)

Q15 · D2 2.2 STRUCTURED-ERROR-RESPONSES / TRANSIENT-ERROR q-2-2-005 · Multi-Agent Research System

CORRECT

A research agent calls an external API via an MCP server. After the 30th query in a batch of 50, the API starts returning HTTP 429 errors. The MCP server returns a generic 'Request failed' for every failure, so the agent abandons the batch after three consecutive failures. What MCP server change would most improve resilience?

- A** Queue all 50 requests at the MCP server level and process them sequentially with built-in rate limiting.

Server-side queuing removes the agent's ability to prioritise, cancel, or reorder queries based on intermediate results. It also creates a long-running blocking call that prevents the agent from doing other work.

- B** Implement automatic retry with exponential backoff inside the MCP server, hiding rate limits from the agent entirely.

Hiding rate limits from the agent removes its ability to make informed decisions, such as prioritising remaining queries, switching to cached results, or reporting partial progress. The agent should receive structured metadata to decide its own recovery strategy.

- C** Return `errorCategory: 'business', isRetryable: false` to tell the agent to stop making requests entirely.

Rate limits are transient (they resolve after a delay), not business errors (which represent policy or rule violations the agent cannot recover from). Marking them as non-retryable causes the agent to abandon 20 remaining queries that would succeed after a brief delay.

- D** Return `errorCategory: 'transient', isRetryable: true`, with a `retryAfterMs` field telling the agent how long to wait.

Rate limiting is a transient error — it will resolve after a delay. Structured metadata with the specific retry delay lets the agent space out remaining queries intelligently rather than abandoning the batch. The agent can continue processing other tasks while waiting.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Transient errors and retry-after)

Q16 · D1 1.1 AGENTIC-LOOPS / ANTI-PATTERNS q-1-1-006 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

A refactoring agent decomposes a legacy Java monolith into microservices. Its loop terminates by parsing the assistant's last message for the phrase 'refactoring complete', running 20 iterations even on simple classes. Logs show `stop_reason` returns 'end_turn' after iteration 4. What is the correct fix?

- A** Replace the natural language parsing with stop_reason inspection: exit on 'end_turn', continue on 'tool_use'.

Parsing natural language for termination is an anti-pattern. The stop_reason field is the authoritative, deterministic signal for agentic loop control. 'tool_use' means Claude has more work to do; 'end_turn' means it considers the task complete.

- B** Add a tool that the agent must call explicitly to signal completion, and check for that tool call each iteration

A custom completion tool adds unnecessary complexity. The API already provides the stop_reason field as a built-in, deterministic mechanism for exactly this purpose.

- C** Improve the phrase detection to also check for 'finished', 'done', and 'all changes applied'

Adding more phrases to parse is still natural language detection, which is an anti-pattern. The model may finish without using any of these phrases, or use them mid-task. The stop_reason field already provides a deterministic signal.

- D** Lower the iteration cap to 5 so the agent stops sooner on simple classes, matching the point where stop_reason already returns 'end_turn'.

Lowering the cap would truncate complex refactoring tasks that genuinely need more iterations, and tuning it to one observed run does not generalise. The root cause is using natural language parsing instead of the stop_reason field for loop control.

WHERE THIS COMES FROM

Lesson 1.1: Agentic Loops (Anti-patterns)

Q17 · D3 3.2 SLASH-COMMANDS-SKILLS / COMMANDS-VS-SKILLS q-3-2-001 ·

Code Generation with Claude Code

CORRECT

A team wants a /review command available to everyone who clones the repository. A developer also wants a personal /brainstorm skill that produces verbose codebase analysis output without cluttering the main conversation. Where should each be created and what configuration does the skill need?

A Both in `.claude/commands/` with the brainstorm skill using context: fork

The `/brainstorm` skill should be personal (not shared with the team), so it should not go in the project-scoped `.claude/commands/` directory.

B Both in `~/.claude/commands/` with instructions for each developer to copy them locally

`~/.claude/commands/` (or equivalently `~/.claude/skills/`) is user-scoped and not shared via version control. Requiring manual copying defeats the purpose of project-scoped configuration.

C `/review` in `.claude/commands/` for team sharing; `/brainstorm` as a `SKILL.md` in `~/.claude/skills/` with context: fork frontmatter

`/review` needs to be project-scoped (`.claude/commands/` or equivalently `.claude/skills/`) so it is shared via version control. Both paths create identical `/commands`. `/brainstorm` is personal, so it goes in `~/.claude/skills/`. context: fork isolates the verbose analysis output from the main conversation.

D `/review` in `CLAUDE.md` as a documented procedure; `/brainstorm` in `.claude/skills/` with allowed-tools restrictions

`CLAUDE.md` is for always-loaded standards, not command definitions. The brainstorm skill needs context: fork for output isolation, not just allowed-tools.

WHERE THIS COMES FROM

Lesson 3.2: Custom Slash Commands and Skills (Where to place commands)

Lesson 3.2: Custom Slash Commands and Skills (Skills system)

Claude Code: Skills ↗

Q18 · D3 3.6 CICD-INTEGRATION / WORKTREE-PARALLEL q-3-6-008 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

The team wants three Claude Code instances working in parallel: one extracting the authentication service, one extracting the billing service, and one updating shared libraries. All three need to commit to the same repository without conflicts. What is the correct setup?

A Use git worktree to create three separate working directories, each on its own branch, with one Claude Code instance per worktree

git worktree creates multiple working directories from the same repository, each checked out to a different branch. Each Claude Code instance operates in its own isolated file system while sharing the same git history. This eliminates file system conflicts between parallel instances.

B Use fork_session to run the three tasks as parallel branches within a single Claude Code session

fork_session creates parallel exploration branches within Claude Code's conversation context, not separate file system environments. Three large extraction tasks each need their own working directory to avoid file conflicts, which requires git worktree.

C Clone the repository three times into separate directories and merge the results manually

Three separate clones create divergent git histories that must be reconciled manually. git worktree provides the same isolation with a shared repository, making merges straightforward through normal branch operations.

D Run all three instances in the same working directory on separate branches, switching branches as needed

Multiple Claude Code instances in the same working directory will cause file system conflicts, dirty working trees, and race conditions, even on different branches. Git branch switching affects the entire working directory.

WHERE THIS COMES FROM

[Anthropic: Claude Code Documentation](#) ↗

[Git: Worktrees](#) ↗

Q19 · D5 5.6 INFORMATION-PROVENANCE / CLAIM-SOURCE-MAPPING q-5-6-002 · Multi-Agent Research System

INCORRECT

A multi-agent research system has three subagents (financial filings, news, technical white papers) and a synthesis agent. Each subagent returns properly attributed findings, but the final synthesised report has no source attribution: stakeholders cannot trace which claim came from which source. Which fix addresses the root cause?

- A Have each subagent include source URLs as inline hyperlinks in their prose output so the synthesis agent can preserve them

Inline hyperlinks in prose are fragile — the synthesis agent will rewrite, merge, and compress prose during summarisation, stripping or disconnecting links from their associated claims. Attribution must be in structured data, not embedded in prose that gets rewritten.

- B Append a bibliography section at the end of the report listing all sources each subagent consulted

A bibliography lists sources but does not map specific claims to specific sources. Stakeholders need to know which claim came from which document, not just which documents were consulted overall. This is document-level attribution, not claim-level provenance.

- C Store all subagent outputs in a database and have the synthesis agent reference database entries by ID instead of incorporating content directly

This adds infrastructure complexity without solving the synthesis problem. The synthesis agent still needs to merge and present findings coherently. Database references do not prevent attribution loss during the summarisation and rewriting that synthesis inherently requires. (your answer)

- D Require subagents to output structured claim-source mappings and instruct the synthesis agent to preserve and merge them.

Structured claim-source mappings survive synthesis because they are data structures, not prose that gets rewritten. The synthesis agent can merge mappings from multiple subagents while preserving the link between each claim and its source. This also enables content-appropriate rendering: financial data as tables, news as prose, technical findings as lists — each with attribution intact. (correct answer)

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Structured claim-source mappings)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Attribution preservation)

Q20 · D3 3.4 PLAN-MODE-EXECUTION / MODE-SELECTION q-3-4-001 · Code Generation with Claude Code

INCORRECT

Your team faces three tasks: (1) restructure a monolith into microservices, (2) fix a null pointer exception in a single function with a clear stack trace, (3) migrate from one logging library to

another across 30 files. Which mode should be used for each?

A Direct execution for all three with comprehensive upfront instructions

Task 1 (monolith restructuring) requires codebase exploration and architectural design. Comprehensive upfront instructions assume you already know the right structure without exploring.

B Plan mode for (1), direct execution for (2) and (3)

Task 3 affects 30 files and requires a consistent migration strategy. Without planning, you risk inconsistent application of the logging library change across files. (your answer)

C Plan mode for (1) and (3), direct execution for (2)

Task 1 involves architectural decisions with multiple valid approaches. Task 3 affects 30 files with a pattern that needs design before execution. Task 2 is a single-function fix with a clear stack trace — direct execution is appropriate. (correct answer)

D Plan mode for all three, since they all involve code changes

Task 2 is a well-understood, limited-scope bug fix. Plan mode would waste time for a change where the problem and solution are already clear.

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Plan vs direct execution)

Lesson 3.4: Plan Mode vs Direct Execution (Plan mode)

Lesson 3.4: Plan Mode vs Direct Execution (Direct execution)

Q21 · D3 3.1 CLAUDE-MD-HIERARCHY / USER-LEVEL q-3-1-015 · Code Generation with Claude Code CORRECT

A developer has personal code formatting preferences (2-space indentation, trailing commas) that differ from some of the projects they work on. They want Claude Code to apply these preferences across all of their projects by default. Where should these preferences be configured?

A In a `.claude/rules/` file without YAML frontmatter so it loads for every session

.claude/rules/ is project-scoped and version-controlled. Personal preferences should not be committed to project repositories where they would affect all team members.

B In the project-level .claude/CLAUDE.md of each repository they work on

Project-level configuration is shared with the team via version control. Personal formatting preferences should not be imposed on the entire team. Additionally, this would require duplicating the preferences across every project.

C In a ~/.claude/skills/ file that is invoked before each coding session

Skills are on-demand workflows invoked for specific tasks, not always-loaded configuration. Formatting preferences need to be applied automatically to every session without manual invocation.

D In ~/.claude/CLAUDE.md, so they apply across all of the developer's projects

User-level CLAUDE.md (~/.claude/CLAUDE.md) applies to all of that developer's projects and is not shared with the team via version control, which is exactly right for personal preferences. Note that CLAUDE.md files are concatenated into context rather than strictly overriding one another, so a project's conflicting standard is not guaranteed to win; if a rule must always be enforced, put it in settings.json or a hook.

WHERE THIS COMES FROM

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (User-level CLAUDE.md)

Claude Code: Memory and CLAUDE.md ↗

Q22 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ACCESS-VS-EMPTY q-2-2-010

INCORRECT

A customer support agent calls a payment processing tool to check a refund status. The tool returns: {"status": "success", "data": [], "message": "No refunds found for this transaction"}. The agent's recovery logic treats this as an error and retries the request. What is wrong with the agent's behaviour?

A The agent should retry more times, as the payment system may be slow to process refund records.

The tool returned status 'success' with a clear message. This is not a slow-processing issue — the query completed successfully and found no refunds. Retrying will produce the same result.

- B** The agent should escalate to a human immediately instead of retrying, as payment queries should never be retried.

There is nothing to escalate. The query succeeded and returned a valid result: no refunds exist. The agent should simply communicate this to the customer. (your answer)

- C** The agent wrongly treats a valid empty result as a failure, not a successful query that found no records.

An empty array result is not a failure when the status indicates success. The tool queried the payment system, found no refund records, and returned that information correctly. The agent should inform the customer that no refund exists for this transaction, not retry. (correct answer)

- D** The tool should not return an empty array — it should return an error with `isRetryable: false` to prevent the retry.

The tool response is correct — a successful query with no results should return an empty array with a success status. The problem is the agent's recovery logic failing to distinguish valid empty results from actual errors.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Access failure vs empty result)

Q23 · D1 1.6 TASK-DECOMPOSITION / DELEGATION q-1-6-008 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

A coordinator agent receives a request to extract a payment processing module from the monolith into a standalone microservice. The task involves creating new API endpoints, migrating database tables, updating 40+ call sites, and writing integration tests. A junior developer suggests the coordinator should handle the entire task itself to avoid subagent communication overhead. Why is this approach wrong?

- A** Loading 40+ files into one context dilutes attention — the coordinator should delegate to scoped subagents instead

Large, multi-file tasks suffer from attention dilution when processed in a single context. Delegating to specialist subagents ensures each agent works with focused context. The coordinator should handle task decomposition and context injection, delegating implementation to specialists.

B The coordinator's API rate limits would be exceeded when processing 40+ files

API rate limits are a practical concern but not the architectural reason to delegate. The root issue is attention dilution — quality degrades when one agent processes too many items in a single context, regardless of rate limits.

C The coordinator should never write code — it should only route tasks

The coordinator can handle simple, well-scoped tasks directly. The issue here is not a blanket rule against the coordinator writing code, but that this specific task is too large and cross-cutting for a single agent to handle effectively.

D The coordinator cannot access file system tools, so it physically cannot make code changes

There is no inherent restriction preventing a coordinator from having file system tools. The issue is architectural — a single agent processing too many files suffers from attention dilution, not tool access limitations.

WHERE THIS COMES FROM

Lesson 1.6: Task Decomposition Strategies (Attention dilution)

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q24 · D2 2.4 MCP-SERVER-INTEGRATION / BUILD-VS-USE q-2-4-001

CORRECT

A team needs to integrate with Jira for issue tracking in their Claude Code workflow. A developer proposes building a custom MCP server. What is the correct first step?

A Evaluate existing community MCP servers for Jira and only build custom if they cannot handle team-specific workflows.

Community servers should always be the first choice for standard integrations. They are maintained, tested, and cover common use cases. Custom builds are justified only when community servers cannot handle team-specific requirements.

- B** Add the Jira integration to ~/.claude.json so each developer can configure it independently.

A team-wide integration should be in project-level .mcp.json so it is version-controlled and shared. User-level ~/.claude.json is for personal/experimental servers.

- C** Build a custom MCP server with the exact API endpoints the team needs.

Building custom is premature. Community MCP servers for Jira already exist and cover standard use cases. Custom builds should be reserved for team-specific workflows.

- D** Use the Jira REST API directly from Bash commands instead of MCP.

Direct API calls bypass the MCP tool interface, losing the benefits of tool descriptions, structured responses, and agent-native integration.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Build-vs-use)

Claude Code: MCP Server Configuration ↗

Q25 · D5 5.4 CODEBASE-EXPLORATION / SCRATCHPAD q-5-4-001 · Code Generation with Claude Code

INCORRECT

A Claude Code agent is exploring an unfamiliar codebase. After investigating several modules, it starts referencing 'typical repository patterns' instead of the specific class names and dependency chains it discovered earlier. What is the most effective mitigation?

- A** Restart the session and ask the agent to explore more efficiently

Restarting loses all accumulated knowledge without addressing the underlying context degradation problem.

- B** Pre-load the entire codebase structure into the initial context before exploration begins

This would consume context budget before exploration even starts and does not address degradation during the session. (your answer)

C Increase the model's context window to accommodate more discovery output

Context degradation is not about running out of tokens — it is about the model losing grip on earlier findings as verbose output accumulates.

D Have the agent maintain scratchpad files recording key findings and reference them for subsequent questions

Scratchpad files persist knowledge across context boundaries, directly counteracting context degradation by keeping critical discoveries accessible. (correct answer)

WHERE THIS COMES FROM

Lesson 5.4: Codebase Exploration and Context Degradation (Context degradation)

Lesson 5.4: Codebase Exploration and Context Degradation (Scratchpad files)

Q26 · D5 5.6 INFORMATION-PROVENANCE / CONFLICT-HANDLING q-5-6-004 · Multi-Agent Research System

CORRECT

A multi-agent research system finds conflicting data on a pharmaceutical compound's efficacy. A 2022 peer-reviewed clinical trial reports 78% efficacy, while a 2024 preprint reports 45% efficacy with a larger sample size. The synthesis agent must produce a recommendation for a medical advisory board. Which approach correctly handles the conflicting sources?

A Present both findings with full provenance: source identity, publication date, peer-review status, sample size, methodology, and any methodological differences that may explain the discrepancy. Let the advisory board weigh the evidence

For a medical advisory board, preserving full provenance is critical. The board needs to weigh peer-review status, sample size, methodology differences, and temporal context. Annotating both sources with complete metadata enables informed human judgment on conflicting scientific evidence.

B Exclude both findings and report that no reliable consensus exists, recommending further research before any conclusions

Excluding available evidence is unhelpful. The advisory board can make informed decisions with properly attributed conflicting data. Withholding findings because they conflict reduces the board's ability to assess the situation.

- C** Use the 2024 preprint as it has a larger sample size and is more recent, noting it supersedes the 2022 study

Recency and sample size alone do not determine reliability. The preprint has not undergone peer review, and simply picking one source discards valuable context that the advisory board needs for informed decision-making.

- D** Average the two values (61.5%) and present it as the best estimate with a note about variance between studies

Averaging conflicting scientific results is methodologically invalid. The studies may have different populations, methodologies, or endpoints. The average has no scientific meaning and obscures the actual disagreement.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflict handling)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflicts intact)

Q27 · D4 4.3 STRUCTURED-OUTPUT / TOOL-CHOICE-FORCING q-4-3-006

INCORRECT

Your data extraction pipeline uses `tool_use` with `tool_choice` set to 'auto'. You need guaranteed structured output for every request, but some documents occasionally produce conversational text responses instead. You have only one extraction tool defined. What is the most reliable fix?

- A** Set `tool_choice` to force the specific extraction tool by name

Setting `tool_choice` to `{type: 'tool', name: 'extract_data'}` forces the model to call that specific tool on every request, guaranteeing schema-compliant structured output with no possibility of text-only responses.

(correct answer)

- B** Switch to prompt-based JSON extraction with strict formatting instructions for more predictable output

Prompt-based JSON is less reliable than `tool_use`. It risks syntax errors and missing fields. Forcing the tool via `tool_choice` is the correct approach. (your answer)

- C** Add a post-processing step that retries any request returning text instead of a tool call

Retrying with the same 'auto' tool_choice may produce the same text response. Fixing tool_choice eliminates the problem at the API level rather than patching it after the fact.

- D** Add a system prompt instruction: 'You must always use the extraction tool and never produce text responses'

System prompt instructions cannot guarantee tool use when tool_choice is 'auto'. The API-level tool_choice parameter is the authoritative mechanism.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (tool_choice modes)

Anthropic: Tool Use Documentation ↗

Q28 · D3 3.5 ITERATIVE-REFINEMENT / EXAMPLES q-3-5-001 · Code Generation with Claude Code **CORRECT**

A developer describes a code transformation in prose. Claude Code interprets it differently each time, producing inconsistent results. What technique should the developer try first?

- A** Provide 2-3 concrete input/output examples showing the exact before and after transformation

Concrete examples eliminate interpretation ambiguity. The model generalises from examples more reliably than from descriptions. This is the documented first-line technique for inconsistent interpretation.

- B** Use the interview pattern to have Claude ask clarifying questions before implementing

The interview pattern is best for unfamiliar domains where the developer might miss considerations. When the developer knows the exact transformation but the model interprets it inconsistently, examples are the direct fix.

- C** Write a test suite first and iterate by sharing test failures

Test-driven iteration is effective but is a more heavyweight approach. Concrete examples are the faster first

step when the issue is inconsistent interpretation of a known transformation.

D Rewrite the prose description with more precise language and technical terminology

More precise prose still relies on the model's interpretation of natural language. If interpretation is inconsistent, more prose will not solve the root cause.

WHERE THIS COMES FROM

Lesson 3.5: Iterative Refinement Techniques (Example-based communication)

Lesson 3.5: Iterative Refinement Techniques (Technique hierarchy)

Q29 · D5 5.3 ERROR-PROPAGATION / ACCESS-VS-EMPTY q-5-3-002 · Multi-Agent Research System

CORRECT

A research subagent queries an academic journal database for articles on renewable energy policy. The database responds successfully but returns zero matching articles. Meanwhile, a second subagent querying a government statistics API receives a connection timeout. The coordinator currently handles both cases identically by retrying three times. What should change?

A Remove retries entirely and immediately escalate both failures to a human operator for investigation

This is disproportionate. The API timeout is a transient failure that retries can resolve. The empty journal result is not a failure at all — it is a valid answer. Neither case requires human escalation.

B Distinguish access failures from valid empty results: retry the API timeout as a transient failure, but accept the zero-match journal response as a valid finding and annotate the coverage gap

Access failures (timeouts, connection errors) indicate the tool could not reach the data source and warrant retry. Valid empty results mean the tool reached the source and found no matches — this IS the answer. Conflating these leads to unnecessary retries on valid results and missed coverage annotations.

C Increase the retry count to five for both cases to give transient failures more time to resolve

More retries do not fix the core problem. The journal database returned a valid empty result — it successfully searched and found nothing. Retrying it wastes time and API calls on an operation that already

completed correctly.

- D** Add exponential backoff to all retries so the subagents wait progressively longer between attempts

Backoff improves retry behaviour for transient failures but does not address the fundamental issue: the journal query does not need retries at all. It returned a valid response indicating no matches exist.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Access failure vs empty)

Lesson 5.3: Error Propagation in Multi-Agent Systems (Local recovery for transient)

Q30 · D4 4.6 MULTI-PASS-REVIEW / SELF-REVIEW-BIAS q-4-6-005 ·

Content Moderation and Classification System

INCORRECT

The moderation team asks a single Claude session to classify a post, then immediately asks the same session to independently review its own classification for quality assurance. The 'review' agrees with the original classification 98% of the time, including cases that human auditors later identify as errors. Why is this self-review ineffective?

- A** The model needs a stronger review prompt with explicit instructions to look for errors in the original classification

Stronger instructions do not overcome the fundamental limitation. The same session retains the reasoning context from the original classification, biasing the review toward agreement.

- B** The same session retains the model's reasoning, so it stays anchored to its classification; use a fresh independent instance to review.

Self-review within the same session is fundamentally limited because the model's conversation context includes the original reasoning. It will naturally be anchored to its prior conclusions. An independent instance with no access to the original decision evaluates the content fresh, providing genuine quality assurance. (correct answer)

- C** The model's temperature is too low, producing deterministic agreement — increase temperature for the review pass

Temperature affects sampling randomness, not reasoning independence. Higher temperature may occasionally produce different outputs but does not create genuine independent review. The model is still

anchored to its own prior reasoning in the same session.

- D** Self-review is effective but the 98% agreement rate simply reflects high initial accuracy — the 2% disagreement is the expected error rate

Human auditors found errors in cases where the self-review agreed, proving the 98% agreement does not reflect accuracy. The self-review is confirming errors, not catching them, due to same-session reasoning bias. (your answer)

WHERE THIS COMES FROM

Lesson 4.6: Multi-Instance Review and Output Validation (Self-review limitation)

Q31 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / CONTEXT-PASSING q-1-3-015 · Multi-Agent Research System **CORRECT**

A coordinator spawns a research subagent to investigate market trends. The subagent returns detailed findings. The coordinator then needs to pass these findings to a synthesis subagent for report writing. However, the synthesis subagent produces a report that contradicts the research findings. What is the most likely cause?

- A** The synthesis subagent has a conflicting system prompt whose instructions override the research data it is handed, replacing it with its own house view of the market.

While possible, conflicting system prompts would produce consistently wrong outputs across all requests. The more common cause is that the coordinator is not passing the full research output, so the synthesis agent fills gaps with its own training data.

- B** The coordinator passes a summary instead of the full structured research output, so the synthesis agent fills gaps from its training data.

Subagents do not share memory. The coordinator must explicitly pass the complete research output to the synthesis agent. If the coordinator only passes a summary, the synthesis agent has no access to the detailed findings and will rely on its training data to fill gaps, potentially contradicting the actual research.

- C** The synthesis subagent has its own web search tool and keeps pulling contradictory information from the open web instead of the research it was handed.

A synthesis subagent should not have web search tools; that violates scoped tool access. The real issue is that the full research output is not being passed to it, not that it is searching the web.

- D** The two subagents run different model versions with separate training data, so they draw opposite conclusions from the same market evidence.

Model version differences would not cause contradictions with explicitly provided research data. The issue is that the research data is not being fully passed to the synthesis agent, not that the agents have different base knowledge.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Context passing)

Lesson 1.3: Subagent Invocation and Context Passing (Structured metadata format)

Q32 · D5 5.4 CODEBASE-EXPLORATION / CRASH-RECOVERY q-5-4-011 · Code Generation with Claude Code

CORRECT

A coordinator spawns four Claude Code subagents to explore a large codebase overnight: one maps the data layer, one traces API routes, one audits tests, one catalogues dependencies. Three hours in, the machine restarts. On rerun, the coordinator starts every exploration again from zero. What design change lets a restarted run continue from where the crash left off?

- A** Run the four explorations sequentially in one session so all findings stay in a single context

Serialising the work gives up parallelism and pushes every verbose discovery into one context window, and a crash still loses the lot: an in-memory context is not durable state, however it is arranged.

- B** Resume the coordinator's previous session so the conversation history is restored

Session resumption restores the coordinator's own conversation, not the subagents' accumulated findings: their contexts were separate and are gone. The coordinator would still have to re-run the explorations to regain that knowledge.

- C** Have each agent export structured state to a manifest the coordinator loads on resume

Structured state exports are the crash-recovery pattern: each agent persists its findings to a known location as it progresses, so after a failure the coordinator loads the manifest, sees what is complete, and seeds the remaining agents with prior findings instead of starting from zero.

D Wrap each subagent in automatic retries so transient failures cannot end the run

Retries help an agent survive a failed tool call; they do nothing for a machine restart that kills the whole run. Recovery needs durable state that outlives the process, not more attempts within it.

WHERE THIS COMES FROM

Lesson 5.4: Codebase Exploration (Crash recovery via structured state manifests)

[Claude Docs: Agent SDK Sessions](#) ↗

Q33 · D3 3.4 PLAN-MODE-EXECUTION / EXPLORE-SUBAGENT q-3-4-007 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

A developer asks Claude Code to 'explain the order processing pipeline' before refactoring it. The pipeline spans 15 classes across 4 packages with deep inheritance and callback chains. Claude Code reads 3 files and produces a superficial summary that misses the callback chain entirely. What approach would produce a more thorough analysis?

A Run multiple Claude Code sessions, each analysing a different package, then manually combine the results

Splitting analysis across sessions loses the cross-package context needed to trace the callback chains and inheritance hierarchies. The pipeline's complexity comes from how the 4 packages interact, which requires a single comprehensive analysis.

B Re-prompt Claude Code with more specific instructions listing all 15 classes to read

This assumes the developer already knows which 15 classes are involved. If the developer knew the full scope, they would not need Claude Code's help understanding the pipeline. The issue is that Claude Code's initial exploration was too shallow.

C Use an Explore subagent for verbose discovery across the pipeline's classes and callback chains

The Explore subagent is purpose-built for verbose, thorough codebase discovery. It systematically follows references, traces inheritance, and maps control flow across files — exactly what is needed for a complex event-driven pipeline. Regular Claude Code defaults to efficient, minimal reads; the Explore subagent prioritises comprehensive discovery.

D Switch to plan mode and ask Claude Code to create a plan for understanding the pipeline

Plan mode can safely explore a codebase before you commit to changes, but it runs in the main conversation and drives toward a change plan. Here the developer needs thorough, verbose discovery of current behaviour, which the Explore subagent isolates and summarises without exhausting the main context, so the Explore subagent is the better fit.

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Explore subagent)

Q34 · D3 3.1 CLAUDE-MD-HIERARCHY / HIERARCHY q-3-1-001 · Code Generation with Claude Code **CORRECT**

Developer A's Claude Code follows the team's API naming conventions perfectly. Developer B, who joined last week, gets inconsistent naming from Claude Code. Both are working on the same repo, same branch. What is the most likely root cause?

A The API naming conventions live in Developer A's user-level CLAUDE.md, not the project-level configuration

User-level CLAUDE.md is not version-controlled or shared via git. Developer A has the instructions locally; Developer B, having just joined, does not. Moving instructions to project-level config (.claude/CLAUDE.md) would fix this.

B Developer B has not run /memory to load the configuration files into the session.

/memory is a debugging command to verify what is loaded, not a command to trigger loading. Configuration files load automatically.

C The conventions are in a .claude/rules/ file that Developer B's editor is not configured to load, so they never reach his session

Rule files in .claude/rules/ are version-controlled and loaded by Claude Code from the repository, not by the editor. On the same repo and branch, both developers get identical rule loading, so an 'editor not loading rules' story cannot explain the difference.

- D** Developer B needs to install an MCP server before the naming convention rules become available to Claude Code.

MCP servers provide tool integrations, not CLAUDE.md configuration. Naming conventions belong in CLAUDE.md files.

WHERE THIS COMES FROM

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (Three-level hierarchy)

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (New-team-member scenario)

Claude Code: Memory and CLAUDE.md ↗

Q35 · D2 2.4 MCP-SERVER-INTEGRATION / RESOURCES q-2-4-012

INCORRECT

A data-analysis agent connects to an MCP server that exposes 40 database tables. Before almost every query, the agent makes several exploratory tool calls to discover which tables exist and what columns they hold, adding latency and token cost to each task. The server author wants to remove this discovery overhead. What is the most effective change?

- A** Add a `describe_schema` tool and instruct the agent in its system prompt to call it before every query.

This keeps an exploratory tool call on every task, which is the exact overhead the author wants to remove. Resources present the catalogue without a per-task call. (your answer)

- B** Expose the table catalogue and column schemas as MCP resources for upfront visibility.

MCP resources expose content catalogues such as database schemas, documentation hierarchies, and issue summaries, giving the agent visibility into available data upfront. The agent no longer needs to call `list_tables` and then `describe_table` for each table, which removes the discovery overhead entirely. (correct answer)

- C** Reduce the server to the five most frequently queried tables so there is less to discover.

This discards capability the agent legitimately needs and still leaves discovery calls for the remaining tables. It treats the symptom, not the discovery mechanism.

- D** Enlarge the agent's context window so it can retain the results of the discovery calls across turns.

A larger context window does not stop the agent making the discovery calls; it still pays the latency and token cost on each task and merely stores the results.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (MCP resources)

Model Context Protocol: Resources ↗

Q36 · D5 5.3 ERROR-PROPAGATION / SILENT-SUPPRESSION q-5-3-005 · Multi-Agent Research System

INCORRECT

A multi-agent system has subagents for web search, academic database, and industry reports. The academic database subagent hits a 403 Forbidden on a specific journal, catches the error, and returns an empty result set marked as successful. The coordinator produces a final report with no academic sources. What is the critical failure in this design?

- A** The subagent silently suppressed the access failure by returning empty results as success, preventing the coordinator from attempting recovery or noting the gap in coverage.

Silent suppression — returning empty results marked as success — is the worst error propagation anti-pattern. The coordinator believes the academic search succeeded and found nothing, so it produces a report without academic sources and without noting the coverage gap. The subagent should have returned structured error context with the failure type, what was attempted, and potential alternatives. (correct answer)

- B** The subagent should have retried the request with exponential backoff, as 403 errors are often transient.

403 Forbidden is a permission error, not a transient error. Retrying will not resolve it — the subagent lacks the required access credentials. The correct response is to propagate the error with its type so the coordinator knows the failure is not retryable.

- C** The entire workflow should have been terminated when the academic database subagent encountered the 403 error.

Workflow termination on a single subagent failure wastes partial results from the web search and industry reports subagents that may have completed successfully. The coordinator should proceed with available results and annotate the coverage gap.

- D The coordinator should validate that all subagents return non-empty results before producing the final report.

Requiring non-empty results from all subagents would cause the workflow to fail when a subagent legitimately finds no matching data. The issue is that the error was suppressed, not that the coordinator failed to validate result volume. (your answer)

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Silent suppression anti-pattern)

Lesson 5.3: Error Propagation in Multi-Agent Systems (Access failure vs empty)

Q37 · D4 4.4 VALIDATION-RETRY / DETECTED-PATTERNS q-4-4-004 ·

Content Moderation and Classification System

INCORRECT

A moderation classification tool includes a `detected\_patterns` array where the model lists specific patterns it identified (e.g. 'repeated slur targeting ethnicity'). Validation checks whether the detected patterns match the assigned category, and on mismatch retries with feedback like 'You classified this as spam but detected patterns of hate speech targeting ethnicity, please re-evaluate.' What is the primary benefit of the `detected\_patterns` field in this workflow?

- A It lets the system auto-correct the category by overriding the model's classification with a rule-based pattern match

The system does not override the model's classification. It feeds back the inconsistency and asks the model to re-evaluate. The model may correct its category or its patterns — the point is self-correction, not rule-based override.

- B It increases classification accuracy by forcing the model to identify specific patterns before assigning a category

While structured reasoning fields can improve accuracy, this framing misses the retry workflow context. The primary benefit is enabling programmatic validation of reasoning consistency and targeted feedback for self-correction. (your answer)

- C** It provides an auditable evidence trail showing which specific content features drove each moderation decision

Auditability is a secondary benefit. The primary benefit in this validation/retry workflow is enabling the system to detect and feed back reasoning inconsistencies for self-correction.

- D** It lets validation detect reasoning inconsistencies and feed back targeted errors the model uses to self-correct.

The detected_patterns field externalises the model's reasoning into structured data that can be programmatically validated against the conclusion. When patterns and category are inconsistent, the specific error feedback ('you detected hate speech patterns but classified as spam') gives the model actionable information to self-correct. This is the core benefit in a retry-with-error-feedback workflow.
(correct answer)

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (detected_patterns fields)

Lesson 4.4: Validation, Retry, and Feedback Loops (Self-correction flow)

Q38 · D2 2.4 MCP-SERVER-INTEGRATION / CONFIG-SCOPE q-2-4-008

INCORRECT

The data platform team needs to share their Snowflake and PostgreSQL MCP server configurations with all team members whilst allowing individual developers to experiment with a personal API integration server. Where should each configuration be placed?

- A** Place all three servers in ~/.claude.json and have each developer copy the configuration manually.

User-level configuration is not version-controlled and requires manual synchronisation. Team-wide servers belong in project-level .mcp.json for consistency and change tracking.

- B** Place the database servers in environment variables and reference them in both .mcp.json and ~/.claude.json.

Environment variables are used for credentials (connection strings, API keys), not for MCP server configuration. The server definitions themselves belong in .mcp.json or ~/.claude.json depending on scope.
(your answer)

- C** Place all three servers in the project-level `.mcp.json` so they are version-controlled and consistent across the team.

Personal experimental servers should not be in project-level configuration. Changes to personal servers would create unnecessary merge conflicts and force experimental integrations on the entire team.

- D** Place the Snowflake and PostgreSQL servers in the project-level `.mcp.json`, and personal API integration servers in user-level `~/.claude.json`.

Project-level `.mcp.json` is version-controlled and shared — correct for team-wide database integrations. User-level `~/.claude.json` is machine-specific — correct for personal experimental servers that should not affect other team members. (correct answer)

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Scoping hierarchy)

Claude Code: MCP Server Configuration ↗

Q39 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / GOAL-ORIENTED-PROMPTS q-1-3-018 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

A coordinator delegates a large refactoring subtask to a specialist subagent, and the architect must write the subagent's prompt. Which prompt design best follows the guide's guidance on how a coordinator should instruct a subagent?

- A** Provide only the high-level goal with no success criteria, so the subagent has maximum freedom

A goal with no quality criteria gives the subagent no way to know when it has actually succeeded. Effective goal-oriented prompts pair the objective with explicit success criteria.

- B** Give an exact step-by-step procedure (open file X, change line 12, run command Y, then edit file Z) so the subagent cannot deviate

A rigid procedure constrains the subagent and breaks the moment reality diverges from the script. The guide specifically warns against procedural instructions in favour of goal-oriented ones.

- C** Omit the goal and instead list the tools the subagent may use, letting it infer the objective from the toolset

Listing tools without stating the goal leaves the objective ambiguous, and the subagent may optimise for the wrong outcome. The prompt must state the goal.

- D** State the goal and its quality criteria (no breaking API changes, all tests pass) and let the subagent choose how to get there.

Goal-oriented prompts specify what to achieve and the quality bar, not a fixed procedure. Pairing the objective with success criteria lets the subagent adapt its approach when it meets something unexpected, which is what the guide recommends.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Goal-oriented prompts)

Q40 · D4 4.3 STRUCTURED-OUTPUT / NULLABLE-FIELDS q-4-3-003 ·

Content Moderation and Classification System

INCORRECT

The moderation system uses `tool_use` with a JSON schema for classification output. All fields including `'sub_category'` and `'target_demographic'` are marked as required. Auditors discover that when a post is spam (which has no target demographic), the model fabricates plausible-sounding demographics like `'general public'` or `'young adults.'` What schema change prevents this fabrication?

- A** Add a prompt instruction telling the model to leave `'target_demographic'` empty whenever a post is spam and the field does not apply.

Prompt instructions cannot override a schema that marks the field as required. The JSON schema enforces the constraint at the API level, and the model must comply with the schema regardless of prompt instructions.

- B** Add a validation step that rejects any `target_demographic` value returned for a post that has been classified as spam, before the record is written.

Post-hoc validation catches the problem but does not prevent it. The model still fabricates values; they are just rejected after the fact. Schema design should prevent fabrication at the source. (your answer)

- C** Make 'target_demographic' nullable so the model can return null when the field does not apply instead of fabricating a value.

Required fields pressure the model to produce a value even when none exists. Making the field nullable gives the model a legitimate way to indicate 'not applicable,' eliminating the incentive to fabricate. This is the standard pattern for fields that apply conditionally. (correct answer)

- D** Remove 'target_demographic' from the schema entirely, on the grounds that its presence is what causes the fabrication in the first place.

Removing the field loses valuable data for categories where target demographic is genuinely relevant (hate speech, harassment). The fix is making it nullable, not removing it.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (Nullable fields)

Q41 · D5 5.1 CONTEXT-WINDOW-MANAGEMENT / POSITION-EFFECTS q-5-1-010 · Multi-Agent Research System

CORRECT

A multi-agent system runs a 45-minute deep research workflow. Around the 30-minute mark, the coordinator's synthesis quality drops: it refers to 'the key findings' instead of specific statistics it cited earlier, and misattributes a claim from the financial subagent to the news subagent. No token limits or errors have been hit. What is the most likely diagnosis and correct mitigation?

- A** The subagents are returning inconsistent data, confusing the coordinator. Add data validation to each subagent's output before it reaches the coordinator

The coordinator previously cited these same findings correctly earlier in the session. The issue is not data quality from subagents but the coordinator's degrading attention to that data over time.

- B** The model is experiencing 'fatigue' from a long session and needs a cooldown period before continuing

LLMs do not experience fatigue. Each inference is independent. The degradation is caused by context window dynamics, not model tiredness.

- C** The token limit has been silently exceeded and the API is truncating early messages. Upgrade to a larger context window model

The question states no token limits have been hit. Context degradation occurs well before token limits are reached — it is an attention quality issue, not a capacity issue.

- D** Context degradation: early results are buried deep in a long context. Consolidate key findings into a structured block near the end.

Context degradation occurs when important information is buried deep in a long conversation. The model's attention to early content diminishes as new content accumulates. Consolidating key findings into a recent, structured block keeps them in the model's effective attention window without losing the specifics.

WHERE THIS COMES FROM

Lesson 5.4: Codebase Exploration and Context Degradation (Context degradation)

Lesson 5.4: Codebase Exploration and Context Degradation (Summary injection)

Q42 · D4 4.2 FEW-SHOT-PROMPTING / FEW-SHOT-AMBIGUITY q-4-2-007

INCORRECT

An invoice-extraction pipeline mislabels fields on documents where the correct interpretation is ambiguous. Which few-shot practices should you apply? (Select 2)

- A** Replace the examples with a longer natural-language description of each field.

Descriptions alone are what produced the inconsistency; examples communicate the expected mapping more reliably than more prose.

- B** Add an example for every document variant seen in production so far.

Volume is not the fix; a handful of targeted examples for the ambiguity beats exhaustive coverage and its context cost. (your answer)

- C** Create two to four targeted examples for the ambiguous cases, showing why one interpretation is chosen.

Targeted examples that demonstrate the reasoning are the guide's specific remedy for ambiguous cases.

- D** Use examples showing correct extraction from varied document structures to fix empty or null extraction of required fields.

Varied-structure examples fix empty or null extraction of fields that are present, which is the few-shot remedy for unreliable extraction; fabrication of absent data is handled by nullable schema fields instead.

(correct answer)

WHERE THIS COMES FROM

Lesson 4.2: Few-Shot Prompting (Few-shot construction)

Anthropic: Multishot (Few-Shot) Prompting ↗

Q43 · D1 1.7 SESSION-STATE-RESUMPTION / FORK-SESSION q-1-7-007 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

A developer is exploring two different strategies for decomposing the monolith's order processing module: (1) splitting by business capability (orders, payments, shipping) or (2) splitting by data ownership (order-db, payment-db, shipping-db). They want to explore both approaches in parallel without losing either analysis. What is the correct Claude Code approach?

- A** Open two terminal tabs and run separate Claude Code sessions with different instructions in each

Separate sessions do not share the baseline context of the current analysis. Both explorations would need to re-do the initial codebase discovery, wasting time and losing the shared foundation.

- B** Use `fork_session` to create two parallel exploration branches from the current session's baseline, one for each decomposition strategy

`fork_session` creates independent branches from a shared baseline. Both branches inherit the current codebase analysis and context but can diverge independently to explore different strategies. Neither branch's exploration affects the other, and both analyses are preserved.

- C** Use `--resume` to alternate between the two strategies in a single session, relying on conversation history to separate the analyses

`--resume` continues a single linear session. Alternating between strategies in one context risks cross-contamination where analysis from one approach influences the other. `fork_session` provides clean isolation for parallel exploration.

- D** Instruct the agent to evaluate both strategies sequentially in the same session, then compare results

Sequential evaluation in a single session means the first strategy's analysis occupies context when exploring the second, potentially biasing the comparison. `fork_session` allows true parallel exploration with isolated contexts.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (`fork_session`)

Lesson 1.7: Session State and Resumption (Session management options)

Q44 · D4 4.5 BATCH-PROCESSING / CUSTOM-ID q-4-5-004

CORRECT

Your CI/CD pipeline processes code reviews using the Message Batches API. After a batch completes, you need to match each review result back to the corresponding pull request. What is the correct mechanism for correlating batch results with their original requests?

- A** Parse the review content to identify which pull request it refers to based on file names mentioned in the review

Content parsing is fragile and error-prone. The Batches API provides a built-in mechanism for result correlation.

- B** Submit each pull request as a separate batch of one item to maintain correlation through the batch ID

Submitting individual batches defeats the purpose of batch processing and does not leverage the `custom_id` mechanism designed for exactly this use case.

- C** Use the `custom_id` field in each batch request to store the pull request identifier, then match results by `custom_id`

The `custom_id` field exists specifically for correlating batch request/response pairs. Setting it to the pull request identifier provides reliable, direct correlation without relying on result ordering or content parsing.

- D** Rely on the batch API returning results in the same order they were submitted

Batch results are not guaranteed to return in submission order. The custom_id field is the correct correlation mechanism.

WHERE THIS COMES FROM

Lesson 4.5: Batch Processing and Prompt Optimisation (custom_id matching)

Anthropic: Message Batches API ↗

Q45 · D4 4.1 SYSTEM-PROMPTS / EXPLICIT-CRITERIA q-4-1-006 ·

Content Moderation and Classification System

CORRECT

The moderation system's prompt instructs Claude to 'be conservative when moderating and err on the side of caution.' Reviewers find that innocuous posts about cooking with knives, news articles about violence, and fictional war stories are all being flagged as policy violations. What is the root cause and fix?

- A Add a second moderation pass that re-reads each flagged post under the same 'be conservative' guidance and strips out the false positives before any action.

A second pass using the same vague 'be conservative' criteria will reproduce the same false positives. Fix the criteria first; add verification layers second.

- B Add an allowlist of safe topics such as cooking, news and fiction that should never be flagged at all by the classifier.

Allowlists are brittle and impossible to maintain comprehensively. New safe topics will constantly appear. The fix is better criteria that teach the model to distinguish context, not a growing list of exceptions.

- C Replace 'be conservative' with explicit categorical criteria defining each violation category with concrete examples.

Vague directives like 'be conservative' give the model no basis for distinguishing between genuine violations and legitimate content that mentions sensitive topics. Explicit criteria with examples of both violations and non-violations (news, fiction, educational content) enable consistent, calibrated decisions.

- D The model is too sensitive, so lower the temperature until it stops over-flagging borderline posts in the moderation queue.

Temperature affects randomness, not the interpretation of moderation criteria. The over-flagging is caused by vague instructions, not sampling behaviour.

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (Explicit criteria over hedge phrases)

Lesson 4.1: System Prompts with Explicit Criteria (Severity calibration)

Q46 · D5 5.3 ERROR-PROPAGATION / STRUCTURED-ERROR-CONTEXT q-5-3-001 · Multi-Agent Research System

CORRECT

A web search subagent in a multi-agent research system times out while researching a complex topic. You need to design how this failure information flows back to the coordinator. Which approach best enables intelligent recovery?

- A Return structured error context including failure type, attempted query, partial results, and potential alternative approaches

This gives the coordinator everything it needs to decide: retry with modified query, try an alternative approach, or proceed with partial results.

- B Catch the timeout and return an empty result set marked as successful, so the rest of the workflow carries on regardless of the failure.

Silent suppression prevents any recovery. The coordinator believes the search succeeded and found nothing, so it will not attempt alternatives.

- C Propagate the timeout exception to a top-level handler that terminates the entire research workflow at once, discarding everything.

Workflow termination wastes partial results from other subagents that may have completed successfully.

- D Implement automatic retry with exponential backoff, returning a generic 'search unavailable' status only after all retries are exhausted

The generic status hides valuable context from the coordinator, preventing informed recovery decisions.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Structured error context)

Q47 · D3 3.2 SLASH-COMMANDS-SKILLS / SKILLS-PLACEMENT q-3-2-014 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

The team creates a reusable `/extract-service` skill that guides Claude Code through the standard steps of extracting a module from the monolith into a microservice (identify boundaries, create service scaffold, migrate code, update call sites, add tests). Where should this skill be stored so that every team member who clones the repository has access?

A In a shared MCP server that all team members connect to

MCP servers provide tool integrations and external data access, not reusable prompt-based workflows. A skill file is the correct mechanism for a guided multi-step procedure.

B In `~/.claude/skills/` on each developer's machine, distributed via the team wiki

`~/.claude/skills/` is user-scoped and not version-controlled. Distributing via wiki requires manual copying and creates drift between developers' versions. The requirement is automatic availability on clone.

C In the repository-root `CLAUDE.md` as an inline procedure

`CLAUDE.md` is for always-loaded standards, not on-demand workflows. An extraction procedure is invoked when needed, not applied to every session. Placing it in `CLAUDE.md` wastes tokens when developers are not extracting services.

D In the repository's `.claude/skills/` directory, committed to version control

`.claude/skills/` is project-scoped and version-controlled. When any team member clones the repository, the skill is automatically available as a `/extract-service` command. Updates to the skill flow through normal git pull, ensuring all developers have the same version.

WHERE THIS COMES FROM

Lesson 3.2: Custom Slash Commands and Skills (Project skill scoping)

Claude Code: Skills ↗

Q48 · D5 5.6 INFORMATION-PROVENANCE / CONFLICT-HANDLING q-5-6-005 · Multi-Agent Research System

CORRECT

A synthesis agent receives findings from three subagents: solar investment grew 15%

(Bloomberg, January 2024), solar investment grew 22% (IEA, March 2024), and cost-per-watt data. The synthesis agent currently picks the IEA figure because it is more recent. What is the correct approach?

- A Always use the most authoritative source (IEA as an international agency) and discard the Bloomberg figure.

Discarding a credible source based on perceived authority silently removes valid data. Both sources are credible, and the difference may reflect different methodologies, scopes, or time periods.

- B Average the two values (18.5%) and cite both sources to present a balanced view.

Averaging conflicting statistics is mathematically misleading. The two figures may measure different scopes (e.g., different regions or investment types), making an average meaningless. The difference needs to be preserved, not merged.

- C Flag the conflicting figures and pause report generation until a human researcher resolves the discrepancy.

Pausing for human resolution is unnecessary when the difference can be explained by temporal or methodological factors. Proper annotation with provenance resolves the ambiguity without blocking the workflow.

- D Present both figures with source attribution and dates, noting what may explain the difference.

Preserving both data points with provenance — source, date, and potential explanations for the difference — lets the consumer interpret the data correctly. Different publication dates and methodologies explain the discrepancy without requiring the synthesis agent to judge which is 'right'.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflict handling)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Temporal awareness)

Q49 · D3 3.4 PLAN-MODE-EXECUTION / PLAN-MODE q-3-4-002 · Code Generation with Claude Code CORRECT

A developer is migrating a monorepo from CommonJS to ES modules: 80+ files, multiple valid strategies, knock-on effects in build tooling, test configuration, and `package.json`. They start in

direct execution mode and begin converting files. Halfway through, the chosen import resolution strategy breaks the test runner. What went wrong?

- A The developer should have used plan mode to explore the codebase and evaluate migration strategies before committing to an approach

An 80+ file migration with multiple valid strategies and cross-cutting concerns (build tooling, test configuration, package.json) is a textbook plan mode scenario. Plan mode would have revealed the test runner incompatibility before any files were changed. Starting with direct execution for a complex, multi-approach task risks costly rework.

- B The developer should have used the interview pattern to have Claude ask questions about the migration before starting

The interview pattern helps surface considerations the developer might miss, but the fundamental issue is choosing direct execution for a task that requires codebase exploration and strategy evaluation. Plan mode, not just questioning, was needed to map dependencies and test impacts.

- C The developer should have provided more detailed upfront instructions to direct execution specifying the exact migration pattern for every file type

The core issue is not insufficient instructions — it is that the developer did not explore the codebase to understand dependencies before choosing a strategy. More detailed instructions would still have used the wrong strategy because the test runner conflict was not known upfront.

- D The developer should have written a complete test suite first and iterated by sharing test failures

Test-driven iteration is an iterative refinement technique (task statement 3.5), not a substitute for upfront planning. The root cause is skipping plan mode for a complex task, not the absence of tests. The existing test runner itself was the source of the incompatibility.

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Plan mode)

Lesson 3.4: Plan Mode vs Direct Execution (Recognising complexity)

Q50 · D4 4.2 FEW-SHOT-PROMPTING / FEW-SHOT-CODED-CONTENT q-4-2-003 ·

Content Moderation and Classification System

INCORRECT

The moderation system classifies hate speech accurately for explicit slurs but misses coded language and dog-whistle terms that human moderators easily recognise. The prompt includes detailed written rules about coded language patterns. What intervention would most improve detection of coded hate speech?

- A Increase the model's context window so it can consider more of the user's post history for context

Context window size is not the bottleneck. The model fails to recognise coded language in individual posts, which is a prompt engineering issue, not a context limitation.

- B Add a comprehensive dictionary of every known coded term and dog-whistle to the system prompt so the classifier can match posts against the full list.

A static dictionary becomes outdated immediately as coded language constantly evolves. It also bloats the prompt without teaching the model to recognise the underlying patterns of coded speech. (your answer)

- C Add 2-4 few-shot examples of coded hate speech, with reasoning naming the coded language and the targeted group.

Few-shot examples with reasoning tags teach the model the pattern-recognition process, not just individual terms. Showing the reasoning chain — surface meaning, coded meaning, targeted group, contextual signals — enables the model to generalise to new coded terms it has not seen before. (correct answer)

- D Add 20+ examples covering every known category of coded hate speech to maximise coverage

Laundry lists of examples dilute attention and teach pattern-matching rather than generalisation. 2-4 targeted examples with reasoning are more effective than 20 examples without reasoning because they teach the analytical process.

WHERE THIS COMES FROM

Lesson 4.2: Few-Shot Prompting (Few-shot for nuanced detection)

Anthropic: Multishot (Few-Shot) Prompting ↗

Q51 · D1 1.2 ORCHESTRATION-PATTERNS / COORDINATOR q-1-2-013 · Multi-Agent Research System CORRECT

In a hub-and-spoke research system, a coordinator manages several search and analysis subagents. Which responsibilities sit with the coordinator rather than with individual subagents?

(Select 3)

- A** Routing all inter-agent communication so subagents never exchange messages directly.

Hub-and-spoke routes every message through the coordinator for observability and consistent error handling.

- B** Running every search itself first so subagents only verify its findings.

The coordinator delegates work rather than duplicating it; doing every search itself defeats the point of decomposition.

- C** Aggregating subagent results and re-delegating targeted follow-up queries when synthesis shows gaps.

Result aggregation and gap-driven re-delegation are coordinator responsibilities in the iterative refinement loop.

- D** Deciding which subagents to invoke based on the complexity of each incoming query.

The guide places dynamic subagent selection with the coordinator, which analyses query requirements instead of always running the full pipeline.

- E** Sharing its full conversation history with each subagent automatically at spawn time.

Subagents run with isolated context; nothing is inherited automatically, and the coordinator passes only the context each task needs.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Lesson 1.2: Multi-Agent Orchestration (Hub-and-spoke architecture)

Q52 · D4 4.3 STRUCTURED-OUTPUT / NULLABLE-FIELDS q-4-3-007

INCORRECT

An extraction pipeline uses `tool\_use` with a 15-required-field schema. For some document types, only 8 of those fields ever appear in the source material. The model consistently fabricates

plausible values for the missing 7, and the instruction 'do not fabricate data' has not helped. Should the team (A) keep all fields required and add post-extraction validation, or (B) redesign the schema? Which is correct and why?

- A Switch to prompt-based JSON extraction where you can instruct the model to omit absent fields entirely

Prompt-based JSON sacrifices schema compliance guarantees. tool_use with optional fields gives both flexibility for absent data and schema validation for present data.

- B Redesign the schema to make the 7 sometimes-absent fields optional/nullable, allowing the model to return null instead of fabricating

Making fields optional/nullable when source data may be absent removes the schema-level pressure to fabricate. The model can legitimately return null for missing information, which is a correct representation of reality. This addresses the root cause at the schema level. (correct answer)

- C Keep all fields required and add post-extraction validation to detect and remove fabricated values

Post-extraction validation cannot reliably distinguish fabricated values from correct ones — fabricated values are designed to look plausible. The schema itself forces fabrication by making all fields required.

- D Create separate schemas for each document type, each containing only the fields present in that document type

Multiple schemas add maintenance complexity and require document type classification before extraction. Making fields nullable in a single schema is simpler and equally effective. (your answer)

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (Nullable fields)

Lesson 4.2: Structured Output with Tool Use (Schema does not prevent fabrication)

Q53 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / STRUCTURED-METADATA q-1-3-001 ·

Multi-Agent Research System

CORRECT

A synthesis agent produces a report with several claims that have no source attribution. The web search and document analysis subagents are working correctly and returning well-sourced

results. What is the most likely root cause?

- A The web search subagent returns prose summaries rather than a structured citation format the synthesis agent can turn into attributions.

The web search subagent is working correctly and returns well-sourced results. The issue is how the coordinator passes those results to the synthesis agent, not the shape the subagent emits.

- B The synthesis agent's system prompt does not instruct it to include citations, so it drops source attribution even when given well-sourced input.

Even with a citation instruction, the synthesis agent cannot attribute claims to sources that were never passed to it. The lesson is explicit: do not blame the synthesis prompt; the root cause is the coordinator dropping structured metadata.

- C The coordinator passes content to the synthesis agent stripped of structured metadata like source URLs and document names.

Context passing must include structured data that separates content from metadata. Without source URLs and document names, the synthesis agent has no attribution information to include.

- D The synthesis agent has no direct web search access, so it cannot re-fetch the sources behind the claims it has been asked to write up.

Giving the synthesis agent web search tools violates the principle of scoped tool access, and re-fetching is not the fix. The coordinator is dropping attribution metadata it already holds.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Structured metadata format)

Lesson 1.3: Subagent Invocation and Context Passing (Context passing)

Q54 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / PARALLEL-SPAWNING q-1-3-010 ·

Content Moderation and Classification System

INCORRECT

A user reports a post that contains both potentially defamatory text and an embedded image that may violate graphic content policies. The coordinator receives this report as a single moderation request. What is the correct delegation strategy?

- A** Route the text to the text classifier and the image to the image analyser in parallel, then aggregate both results for the final decision.

The coordinator should decompose multi-concern requests into independent subtasks. Text defamation and image policy violations are separate analysis dimensions that can be evaluated in parallel. The coordinator aggregates the results to make a unified moderation decision. (correct answer)

- B** Triage the whole report to whichever subagent handles the more severe category, so the moderation decision reflects the worst violation present.

Severity cannot be determined until both analyses are complete, so there is nothing to triage on. Routing to a single subagent also means one concern goes unanalysed.

- C** Spawn a new combined text-and-image subagent for multi-modal reports so one specialist owns the whole decision rather than splitting it in two.

Creating a new combined subagent duplicates capability that already exists in the specialist subagents. The coordinator's role is to decompose and delegate, not to spawn new agents for every combination of concerns. (your answer)

- D** Send the entire report to the text classifier first, then forward its output to the image analyser so the image step runs only after the text step has finished.

Sequential processing creates unnecessary latency when the two concerns are independent. The text and image analyses do not depend on each other and can run in parallel.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning)

Lesson 1.4: Workflow Enforcement and Handoff (Multi-concern request handling)

Q55 · D1 1.2 ORCHESTRATION-PATTERNS / ITERATIVE-REFINEMENT q-1-2-012 · Multi-Agent Research System

CORRECT

In a multi-agent research system the coordinator delegates to search and analysis subagents, then invokes a synthesis subagent. Reviewing the synthesis, the coordinator finds two claims with no supporting evidence and one sub-question left unanswered. What is the coordinator's correct next step?

- A** Tell the synthesis subagent to fill the two evidence gaps and the open sub-question from its own knowledge, without any new delegation.

The synthesis subagent has no tools or sources to gather new evidence. Asking it to invent the missing support produces exactly the unsourced claims the review is meant to catch; new evidence requires re-delegation.

- B** Accept the synthesis as final, since re-running the search and analysis subagents would push the coordinator past its configured iteration budget.

Shipping unsupported claims to save iterations defeats the purpose of the review. The iteration cap is a runaway safety net, not a reason to leave known gaps unfilled.

- C** Assess the synthesis for gaps, re-delegate targeted follow-up queries to fill them, then re-invoke synthesis with the new findings.

The coordinator owns iterative refinement: assess the synthesis for gaps, re-delegate focused follow-up queries, then re-synthesise. This closes the evidence gaps without discarding the work already done.

- D** Restart the whole pipeline from scratch with a fresh coordinator to avoid contaminated context

A full restart throws away correct findings and repeats work. Targeted re-delegation of only the missing pieces is the efficient, guide-recommended fix.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q56 · D4 4.4 VALIDATION-RETRY / RETRY-BOUNDARY q-4-4-003 ·

Content Moderation and Classification System

INCORRECT

The moderation system validates that every classification includes a 'reasoning' field explaining the decision. When validation fails (empty reasoning), the system retries. For most posts, the retry succeeds after feeding back the error 'reasoning field was empty — provide a brief justification for the classification.' However, for posts in unfamiliar languages, retries consistently fail. What should the system do?

- A** Retry format errors on analysable posts; route capability gaps like unfamiliar languages to human review.

The retry boundary distinguishes between fixable format issues and unfixable capability gaps. Empty reasoning on analysable posts is a format error that retries with error feedback can fix. Unfamiliar languages represent absent capability — no amount of retrying will create language understanding. Route these to human review. (correct answer)

- B** Add the unfamiliar language to the system prompt as a supported language to encourage the model to try harder

Listing an unsupported language in the prompt does not give the model language understanding. This may actually worsen the problem by encouraging fabricated analysis of content the model cannot read.

- C** Increase the retry count from 1 to 5 for unfamiliar language posts since the model may succeed with more attempts

More retries are ineffective when the model lacks the capability to analyse content in that language. The failure is not a transient format issue — the model genuinely cannot provide reasoning for content it cannot understand.

- D** Remove the reasoning validation requirement for posts in unfamiliar languages

Removing the validation requirement means these posts would receive classifications without any justification, which undermines auditability. The correct approach is human review, not lower standards. (your answer)

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry boundary)

Q57 · D1 1.4 WORKFLOW-ENFORCEMENT-HANDOFF / CONFIDENCE-ROUTING q-1-4-007 ·

Content Moderation and Classification System

INCORRECT

The moderation system auto-removes posts classified as policy violations. Production logs reveal that 3% of auto-removed posts were legitimate content (false positives), generating user complaints and eroding platform trust. The team considers two approaches: (A) escalate all borderline cases to human reviewers, or (B) add a confidence threshold where only high-confidence violations are auto-removed and everything else goes to human review. Which approach is better and why?

- A** Approach B: auto-remove only high-confidence violations, escalate uncertain cases to human review, and auto-approve high-confidence safe content.

Confidence-based routing creates a tiered system: high-confidence violations are actioned immediately, high-confidence safe content passes through, and uncertain cases receive human review. This balances speed, accuracy, and reviewer workload while directly addressing the false positive problem. (correct answer)

- B** Neither — instead lower the classification threshold so fewer posts are flagged as violations

Lowering the threshold reduces false positives but increases false negatives (genuine violations going undetected). This trades one problem for another rather than adding an appropriate escalation path.

- C** Approach A: escalate every borderline case, because human reviewers are more accurate than the classifier and a 3% false-positive rate is a trust problem.

Escalating all borderline cases overwhelms the human review queue without leveraging the system's ability to handle clear-cut cases autonomously. The goal is to route intelligently, not to defer everything. (your answer)

- D** Approach B, but apply a single fixed 95% confidence threshold to every category regardless of how costly a false positive is for that content type.

A single fixed threshold across all categories ignores that violation types have different risk profiles. The threshold should be calibrated per category based on the cost of false positives versus false negatives.

WHERE THIS COMES FROM

Lesson 1.4: Workflow Enforcement and Handoff (Confidence-based escalation)

Lesson 1.4: Workflow Enforcement and Handoff (Enforcement spectrum)

Q58 · D3 3.3 PATH-SPECIFIC-RULES / PATHS-FRONTMATTER q-3-3-008 · Code Generation with Claude Code

CORRECT

A large TypeScript monorepo has a root CLAUDE.md with universal coding standards and a .claude/rules/ directory with topic-specific rule files. A developer notices that conventions from testing.md in .claude/rules/ are loading even when editing API handler files, consuming unnecessary tokens. The testing.md file has no YAML frontmatter. What should the developer do

to fix this?

- A Move testing.md out of .claude/rules/ and into a directory-level CLAUDE.md inside the test folder

Test files are co-located with source files throughout 200+ packages in this monorepo. A directory-level CLAUDE.md in a single test folder would not cover test files in other directories, and creating one in every directory would be unmaintainable.

- B Use an @./claude/rules/testing.md line in the root CLAUDE.md so the file only loads when the import is reached

@ path imports load eagerly when the CLAUDE.md is loaded — the imported file's content is inlined at that moment, regardless of which file the developer is editing. They are a source-organisation mechanism, not a conditional-loading mechanism. Path-scoped frontmatter in .claude/rules/ is the only way to make rule files load only for matching paths.

- C Add the testing conventions to the root CLAUDE.md instead, since rules files cannot be path-scoped

Rules files absolutely support path scoping via YAML frontmatter with a paths field. Moving conventions to root CLAUDE.md would make the token efficiency problem worse, not better, because root CLAUDE.md loads for every session.

- D Add YAML frontmatter with a paths field of test-file globs to testing.md so it loads only for test files

Without YAML frontmatter, rule files in .claude/rules/ load for all sessions. Adding a paths field with glob patterns restricts loading to sessions where the developer is editing matching files. This is the correct approach for token-efficient, conditional convention loading.

WHERE THIS COMES FROM

Lesson 3.3: Path-Specific Rules (Path-specific rules)

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (.claude/rules/ directory)

Q59 · D5 5.2 ESCALATION-AMBIGUITY / EXPLICIT-ESCALATION q-5-2-005

INCORRECT

A customer says: 'This is ridiculous, I have been waiting 20 minutes. Just connect me to a real person.' The agent has access to the customer's account and can see the issue is a simple

password reset that takes 30 seconds to resolve. What should the agent do?

- A Acknowledge the frustration and offer to resolve the password reset, explaining it will take only 30 seconds — the issue is straightforward and escalation is unnecessary.

The customer has explicitly requested a human agent. Regardless of how simple the issue is, the agent must honour the explicit request immediately. Attempting to resolve first ignores the customer's stated preference. (your answer)

- B Ask the customer if they would prefer to wait for a human or let the agent resolve the password reset quickly.

The customer has already stated their preference clearly. Asking again ignores their explicit request and adds friction to an already frustrating experience.

- C Escalate to a human agent immediately without attempting to resolve the issue, because the customer explicitly requested a human.

When a customer explicitly says 'connect me to a real person', the agent must escalate immediately. This is one of the three valid escalation triggers. The simplicity of the issue is irrelevant — the customer's explicit request takes priority. (correct answer)

- D Run sentiment analysis to confirm the customer is genuinely frustrated before deciding whether to escalate or attempt resolution.

Sentiment analysis is an unreliable escalation trigger. More importantly, the customer's explicit request for a human is the trigger here, not their emotional state. Sentiment analysis adds unnecessary processing.

WHERE THIS COMES FROM

Lesson 5.2: Escalation and Ambiguity Resolution (Valid escalation triggers)

Lesson 5.2: Escalation and Ambiguity Resolution (Frustration nuance)

Q60 · D4 4.1 SYSTEM-PROMPTS / SEVERITY-EXAMPLES q-4-1-007 ·

Content Moderation and Classification System

CORRECT

A moderation team introduces a severity scale ('low', 'medium', 'high', 'critical'). The same post (a user threatening to 'destroy' a competitor's product in a review) is classified as 'high' on some runs and 'low' on others. The prompt defines severities with phrases like 'high means the content

is clearly harmful' and 'low means the content is mildly inappropriate'. What should they change?

- A** Remove the severity scale and use binary classification (violation / not violation) to reduce inconsistency

Binary classification loses valuable granularity needed for routing decisions (auto-remove vs escalate vs warn). The fix is better severity criteria, not removing severity entirely.

- B** Set the temperature to 0 to eliminate classification variance from one run of the pipeline to the next.

Lower temperature reduces randomness but does not fix ambiguous criteria. The model is inconsistent because the severity definitions are vague, not because of sampling variance.

- C** Replace the prose severity descriptions with a concrete example per level as a calibration anchor.

Concrete examples anchor each severity level to specific, observable patterns. When the model sees 'destroy a competitor's product' in a review context, it can match against the 'low severity — figurative language' example rather than interpreting 'clearly harmful' differently each run.

- D** Run the classification three times over and take the majority vote to settle the severity for each item.

Majority voting averages out noise but triples cost and latency without fixing the root cause. With vague criteria, the model may consistently misclassify in the same direction, making voting ineffective.

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (Severity calibration)

SUPPORT

Found this useful?

The site is free and always will be. If the mock exam helped, you can buy me a coffee. Support goes to keeping the lights on.



BUY ME A COFFEE